

p1044R0

std::async() in an Executors World

Published Proposal, 4 May 2018

Authors:

[Thomas Rodgers](#) (RedHat)

[H Carter Edwards](#) (Nvidia)

Mark Hoemmen (Sandia National Laboratories)

Audience:

SG1, LEWG

Project:

ISO JTC1/SC22/WG21: Programming Language C++

Abstract

We show an evolutionary path for code written to std::async() to leverage Executors

Table of Contents

1	Background
2	An Execution Context for async()
3	Proposed wording for Standard Async Execution Context and Executor
4	Transliteration of async()
5	Open Questions
	References
	Informative References

§ 1. Background

C++11 introduced `async()` along with several other basic concurrency primitives. Many regard `async()` as fundamentally broken [\[n3637\]](#) and in need of deprecation [\[n3777\]](#). Following much discussion and a passionate plea [\[n3780\]](#), the effort to deprecate `async()` prior to an executors-based replacement was abandoned. The guidance from SG1 to the authors of the various Executors proposals, since at least the 2014 Urbana-Champaign SG1 discussion of [\[n4242\]](#) (see: [\[SG1-Mins-n4242\]](#)), has been that any Executors proposal must clearly address the deficiencies of `async()`.

At the Fall 2017 Albuquerque meeting, a companion paper [\[p0737\]](#) to the Executors proposal was presented that more fully defines the notion of an `execution_context`. The paper includes a proposal for a formal definition of an "executor for `async()`" and its supporting library machinery. This offers an evolutionary path away from the undesirable, but possibly widely used at this point, semantics of the current `async()`.

At the Winter 2018 Jacksonville meeting, there was clear direction to the authors of the Executors proposal [\[p0443\]](#) to provide library usage examples (see: [\[LEWG-Mins-p0443\]](#)). This paper intends to address this request by drawing on [\[p0737\]](#) as it specifically relates to `async()`.

The current wording on the behavior of `std::async` is as follows -

— If `launch::async` is set in policy, calls `INVOKE(DECAY_COPY(std::forward<F>(f)), DECAY_COPY(std::forward<Args>(args))...)` (23.14.3, 33.3.2.2) as if in a new thread of execution represented by a thread object with the calls to `DECAY_COPY` being evaluated in the thread that called `async`. Any return value is stored as the result in the shared state. Any exception propagated from the execution of `INVOKE(DECAY_COPY(std::forward<F>(f)), DECAY_COPY(std::forward<Args>(args))...)` is stored as the exceptional result in the shared state. The thread object is stored in the shared state and affects the behavior of any asynchronous return objects that reference that state.

— If `launch::deferred` is set in policy, stores `DECAY_COPY(std::forward<F>(f))` and `DECAY_COPY(std::forward<Args>(args))...` in the shared state. These copies of `f` and `args` constitute a deferred function. Invocation of the deferred function evaluates `INVOKE(std::move(g), std::move(xyz))` where `g` is the stored value of `DECAY_COPY(std::forward<F>(f))` and `xyz` is the stored copy of `DECAY_COPY(std::forward<Args>(args))...`. Any return value is stored as the result in the shared state. Any exception propagated from the execution of the deferred function is stored as the exceptional result in the shared state. The shared state is not made ready until the function has completed. The first call to a non-timed waiting function (33.6.5) on an asynchronous return object referring to this shared state shall invoke the deferred function in the thread that called the waiting function. Once evaluation of `INVOKE(std::move(g), std::move(xyz))` begins, the function is no longer considered deferred. [Note: If this policy is specified together with other policies, such as when using a policy value of `launch::async | launch::deferred`, implementations should defer invocation or the selection of the policy when no more concurrency can be effectively exploited. — end note]

— If no value is set in the launch policy, or a value is set that is neither specified in this document nor by the implementation, the behavior is undefined.

The language run-time currently provides *hidden* executor types which fulfill the requirements of these launch policies. In particular, a launch policy of `launch::async` requires that the run-time provide a new single thread executor to evaluate the supplied Callable. The policy for `launch::deferred` most resembles the concept of an *inline executor*, which only evaluates the supplied Callable when the returned `std::future` is awaited.

[p0737] seeks the formal introduction of an *execution context* concept, which [p0443] currently leaves unspecified. [p0737] defines the `ExecutionContext` as follows -

A concurrency and parallelism **execution context** manages a set of **execution agents** on a set of **execution resources** of a given **execution architecture**.

These execution agents execute work, implemented by a callable, that is submitted to the execution context by an **executor**.

One or more types of executors may submit work to the same execution context. Work submitted to an execution context is **incomplete** until (1) it is invoked and exits execution by return or exception or (2) its submission for execution is canceled.

Note: The *execution context* terminology used here and in the Networking TS [\[n4656\]](#) deviate from the traditional *context of execution* usage that refers to the state of a single executing callable; *e.g.*, program counter, registers, stack frame.

This ExecutionContext concept is specified as follows -

```

class ExecutionContext /* exposition only */ {
public:

    template <typename ExecutionContextProperty>
        /* exposition only */ detail::query_t< ExecutionContext , ExecutionContextProperty>
        query(ExecutionContextProperty p) const ;

    ~ExecutionContext();

    // Not copyable or moveable
    ExecutionContext( ExecutionContext const & ) = delete ;
    ExecutionContext( ExecutionContext && ) = delete ;
    ExecutionContext & operator = ( ExecutionContext const & ) = delete ;
    ExecutionContext & operator = ( ExecutionContext && ) = delete ;

    // Execution resource
    using execution_resource_t = /* implementation defined */ ;

    execution_resource_t const & execution_resource() const noexcept ;

    // Executor generator
    template< class ... ExecutorProperties >
        /* exposition only */ detail::executor_t< ExecutionContext , ExecutorProperties... >
        executor( ExecutorProperties... );

    // Waiting functions:
    void wait();
    template< class Clock , class Duration >
        bool wait_until( chrono::time_point<Clock,Duration> const & );
    template< class Rep , class Period >
        bool wait_for( chrono::duration<Rep,Period> const & );
};

bool operator == ( ExecutionContext const & , ExecutionContext const & );
bool operator != ( ExecutionContext const & , ExecutionContext const & );

// Execution context properties:

constexpr struct reject_on_destruction_t {} reject_on_destruction;
constexpr struct abandon_on_destruction_t {} abandon_on_destruction;

```

```
constexpr struct abort_on_destruction_t {} abort_on_destruction;
constexpr struct wait_on_destruction_t {} wait_on_destruction;
```

§ 2. An Execution Context for `async()`

[p0737] Proposes that there exists in the underlying runtime, a *hidden* execution context, into which **33.6.9 Function template `async`** launches asynchronous work, and proposes that this execution context be exposed along with the *hidden* executors fulfilling `std::async` launch policies. Exposing this execution context and these executor types enables current use of `std::async` to be transliterated to the executor model as follows.

```
// Equivalent without- and with-executor async statements without launch policy
auto f = std::async( []{ std::cout << "anonymous way\n" } );
auto f = std::async( std::async_execution_context::executor() , []{ std::cout << "anonymous way\n" } );

// Equivalent without- and with-executor async statements with launch policy
auto f = std::async( std::launch::deferred , []{ std::cout << "anonymous way\n" } );
auto f = std::async( std::async_execution_context::executor( std::launch::deferred ) , []{ std::cout << "anonymous way\n" } );
```

Asynchronous execution behavior is unchanged following such a well-defined transliteration. The use of `std::async` is now prepared, at the developers' discretion, to replace the leading executor argument with a different executor.

§ 3. Proposed wording for Standard Async Execution Context and Executor

The proposed wording is extracted from [p0737] with modifications for a minimal proposal and to align with executors proposal [p0443].

```

namespace std {
namespace experimental {
inline namespace executors_v1 {
namespace execution {

    namespace launch {
        struct async_t {
            static constexpr bool is_requirable = true;
            static constexpr bool is_preferable = true;
        };

        struct deferred_t {
            static constexpr bool is_requirable = true;
            static constexpr bool is_preferable = true;
        };

        constexpr async_t async; // executor property
        constexpr deferred_t deferred; // executor property
    } // namespace launch

    class async_executor_t ; // implementation defined

    class async_execution_context_t {
    public:

        // Not copyable or moveable
        async_execution_context_t( async_execution_context_t const & ) = delete;
        async_execution_context_t( async_execution_context_t && ) = delete ;
        async_execution_context_t & operator = ( async_execution_context_t const & ) = delete;
        async_execution_context_t & operator = ( async_execution_context_t && ) = delete;

        // Executor generator
        template<class ... ExecutorProperties>
            /* exposition only */ detail::executor_t< ExecutionContext , ExecutorProperties>
            executor( ExecutorProperties ... p );

        // Execution resource - deferred
        // Waiting - deferred
    };

    // Execution context into which std::async launches work
    extern async_execution_context_t async_execution_context;

```

```

} // namespace execution
} // inline namespace executors_v1
} // namespace experimental
} // namespace std

namespace std {
    template<class Function , class ... Args>
    future<std::result_of<std::decay_t<Function>(std::decay_t<Args>...)>>
    async( std::experimental::execution::async_executor_t exec , Function &&
} // namespace std

```

```

experimental::execute::launch::async;
experimental::execute::launch::deferred;

```

Executor properties isomorphic with `std::launch` launch policies of the same name.

```
class async_executor_t;
```

An implementation-defined type satisfying `TwoWayExecutor` requirements [\[p0443\]](#).

```
class async_execution_context_t;
```

An implementation-defined type partially satisfying execution context requirements [\[p0737\]](#).

Specification of execution context properties, construction, destruction, and wait functionality is deferred.

```

template< class ... ExecutorProperties >
    /* exposition only */ detail::executor_t< ExecutionContext , ExecutorProp
    async_execution_context_t::executor( ExecutorProperties ... p );

```

Returns: A *executor* with `*this` *execution context* and execution properties `p`. If `p` is empty, is `std::experimental::execution::launch::async`, or is `std::experimental::execution::launch::deferred` the *executor* type is `async_executor_t`.

```
extern async_execution_context_t async_execution_context;
```

Global execution context object enabling the equivalent invocation of callables through the with-`async_executor_t` `std::async` and without-executor `std::async`. Guaranteed to be initialized during or before the first use.

```
template< class Function , class ... Args >
future<std::result_of<std::decay_t<Function>(std::decay_t<Args>...)>>
async( async_executor_t exec , Function && f , Args && ... args );
```

Effects: If `exec` has an `std::experimental::execution::launch` property then equivalent to invoking `std::async( policy , f , args... );` with `policy` of the same name; otherwise equivalent to invoking `std::async( f , args... );` Equivalency is symmetric with respect to the current, non-executor `std::async` functions.

§ 4. Transliteration of `async()`

With this proposal, transliteration of current `std::async` usage to the executor model is as follows.

```
// Equivalent without- and with-executor async statements without launch po

auto f = std::async( []{ std::cout << "anonymous way\n"} );
auto f = std::async( std::experimental::execution::async_execution_context.

// Equivalent without- and with-executor async statements with launch polic

auto f = std::async( std::launch::deferred , []{ std::cout << "anonymous wa
auto f = std::async( std::experimental::execution::async_execution_context.
```

§ 5. Open Questions

1. [\[p0737\]](#) does not propose a mechanism to migrate the executor aware `std::async` template from `std::experimental::execute::async_executor_t` to other executors types.
2. 33.6.2 currently specifies `std::launch::async` and `std::launch::deferred` as:

```
enum class launch : unspecified {
    async = unspecified ,
    deferred = unspecified ,
    implementation-defined
};
```

33.6.2 further constrains `std::launch` to be a *bitmask* with the following note - [Note: Implementations can provide bitmasks to specify restrictions on task interaction by functions launched by `async()` applicable to a corresponding subset of available launch policies. Implementations can extend the behavior of the first overload of `async()` by adding their extensions to the launch policy under the “as if” rule. -- end note]

In order to preserve backward compatibility with existing usage of `std::launch` this would imply that the `std::experimental::execute::launch` property types would also need to behave as-if they were a bitmask. This implies that `std::experimental::execution::launch::async` | `std::experimental::execution::launch::deferred` return another viable property type that represents the union of these two launch properties. Another option is for [p0443] to be revised to allow executor properties to be implemented as `enum class` or `constexpr` values potentially of the same type.

3. [n4406] and [p0761] suggest a `.on()` member taking an executor to allow parallel algorithm launch policies to compose with executors. Do the policies of `std::launch` differ enough, conceptually, from the launch policies specified in 23.19.2 to warrant a different mechanism for composition with executors?

§ References

§ Informative References

[LEWG-Mins-p0443]

Minutes of LEWG discussion of [p0443] at 2018 Jacksonville WG21 meeting.

[N3637]

Herb Sutter; Chandler Carruth; Niklas Gustafsson. [async and ~future \(Revision 3\)](https://wg21.link/n3637). April 17, 2013. URL: <https://wg21.link/n3637>

[N3777]

Herb Sutter. [Wording for deprecating async](https://wg21.link/n3777). September 23, 2013. URL: <https://wg21.link/n3777>

[N3780]

Nicolai Josuttis. [Why Deprecating async\(\) is the Worst of all Options](https://wg21.link/n3780). September 26, 2013. URL: <https://wg21.link/n3780>

[N4242]

Chris Kohlhoff <chris@kohlhoff.com>. [Executors and Asynchronous Operations](https://wg21.link/n4242). October 13, 2014. URL: <https://wg21.link/n4242>

[N4406]

Jared Hoberock; Michael Garland; Olivier Giroux. [Parallel Algorithms Need Executors](https://wg21.link/n4406). April 10, 2015. URL: <https://wg21.link/n4406>

[N4656]

[Working Draft, C++ Extensions for Networking](https://wg21.link/n4656). 2017-03-17. URL: <https://wg21.link/n4656>

[P0443]

Jared Hoberock; et al. [A Unified Executors Proposal for C++](https://wg21.link/p0443). February 12, 2018. URL: <https://wg21.link/p0443>

[P0737]

H. Carter Edwards; et al. [Execution Context of Execution Agents](https://wg21.link/p0737). URL: <https://wg21.link/p0737>

[P0761]

Jared Hoberock; et al. [Executors Design Document](https://wg21.link/p0761). February 12, 2018. URL: <https://wg21.link/p0761>

[SG1-Mins-n4242]

Minutes of SG1 discussion of [n4242] at 2014 Urbana-Champaign WG21 meeting.